

猫眼娱乐前端一面

猫眼娱乐前端一面

【基础题】

1. flex 布局：情景题 考察使用flex=1实现自适应宽度
2. 移动端的适配问题 是否具有相关的经验
3. 重排重绘
4. 了解哪些浏览器缓存？
5. tcp vs udp（可靠性/传输效率）；tcp更可靠的实现策略（流量控制/拥塞控制.）
6. http vs httpswebpack 热更新原理
7. tree shaking是否了解？有什么前置条件（需要适用于什么场景？）？

基于什么规范（原理）怎么判定是否使用了-es6的规范，在编译前需要进行静态

【react相关】

8. jsx 和js区别？jsx为什么只有允许有一个父节点useState vs 类组件的setSateuseMemo

如何对值实现数据缓存

9. 【设计模式】熟悉什么设计模式？我回答 发布订阅，面试官要求 手写伪代码

【算法题】

10. 最长回文子串

参考答案：

1. Flex 布局：使用 `flex: 1` 实现自适应宽度

Flexbox 是 CSS 的一种布局模型，允许你通过简单的属性设置，实现元素在容器中的对齐和分布。

`flex: 1` 是一种常用的方式，表示元素可以相对于其他元素扩展或收缩，占据可用的剩余空间。

```
1 .container {
2   display: flex;
3   width: 100%; /* 父容器的宽度 */
4 }
5
6 .box1 {
7   flex: 1; /* box1 占据剩余空间 */
8 }
9
10 .box2 {
11   flex: 2; /* box2 占据的空间是 box1 的两倍 */
12 }
13
```

解释：`flex: 1` 的意思是该元素将按比例占用父容器中可用的剩余空间。`flex` 属性等同于 `flex-grow`、`flex-shrink` 和 `flex-basis` 的简写，其中 `flex-grow` 定义元素如何扩展，而 `flex-shrink` 则定义元素在空间不足时如何缩小。

2. 移动端适配问题

移动端适配主要目的是让网页在不同尺寸和分辨率的设备上都能有良好的显示效果。以下是几个常用的适配方案：

- **Viewport 设置：**使用 `<meta>` 标签来控制移动端浏览器的页面缩放和布局。

```
1 <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- **媒体查询：**使用 `@media` 查询根据设备屏幕尺寸设置不同的样式。

```
1 @media only screen and (max-width: 600px) {
2   .container {
3     flex-direction: column;
4   }
5 }
```

- **响应式单位：**尽量使用百分比（%）、`em`、`rem`、`vh` 和 `vw` 等相对单位，而非固定像素值，这样可以根据设备自动调整元素的大小。
- **灵活图片和字体：**使用 `max-width: 100%` 来确保图片不会超过父容器宽度，字体大小也可用 `vw` 来根据屏幕宽度缩放。

3. 重排（Reflow）和重绘（Repaint）

重排（Reflow）和 **重绘**（Repaint）是浏览器渲染页面时的两个主要过程：

- **重排**：当页面的布局或结构发生改变时，浏览器需要重新计算所有元素的尺寸、位置等。常见的触发重排的操作包括：
 - 改变元素的宽高、边距或填充（如 `width`，`height`，`margin`，`padding`）。
 - DOM 节点的增删，或改变 DOM 树结构。
 - 修改 `display`，`position`，`float` 等布局属性。
- **重绘**：重绘发生在元素的外观改变时，但不影响布局，比如改变背景色、文字颜色、阴影等。这比重排的开销小，但也可能导致性能问题。

优化方法：

- 避免频繁改变布局属性，尽量将样式操作批量处理。
- 使用 `class` 切换样式而非逐个操作 DOM 属性。
- 利用 `requestAnimationFrame` 和 `debounce` 来降低频繁的布局变化。

4. 浏览器缓存类型

常见的浏览器缓存策略主要有以下几种：

- **HTTP 缓存**：通过 HTTP 头来控制缓存的行为，常用的控制指令包括：
 - `Cache-Control`：设置缓存策略，常见值有 `no-cache`，`no-store`，`max-age=3600`（一小时）。
 - `ETag`：为资源生成唯一标识符，浏览器每次请求时会验证资源是否改变。
 - `Expires`：设定资源的过期时间。
- **本地存储（localStorage）**：保存数据到浏览器本地，数据不会随页面刷新或关闭而消失。
- **会话存储（sessionStorage）**：与 `localStorage` 类似，但数据仅在页面会话期间有效，关闭页面后数据就会消失。
- **IndexedDB**：浏览器提供的更高级的存储机制，允许存储复杂的数据结构，如对象和文件。
- **Cookie**：小容量数据存储，通常用于会话跟踪和用户认证。

5. TCP 与 UDP

- **TCP（传输控制协议）**：
 - 面向连接，建立可靠的连接，保证数据传输的完整性。
 - 有序、可靠，确保数据不会丢失、重复或错序。
 - 实现了**流量控制**（控制发送端数据传输的速度）和**拥塞控制**（防止网络过载）。

- TCP 通过三次握手建立连接，通过四次挥手断开连接。
- **UDP**（用户数据报协议）：
 - 无连接，不保证数据的可靠传输。
 - 传输效率高，适合要求低延迟的场景，如视频流、实时游戏等。

总结：TCP 适用于需要高可靠性的场景，如文件传输、邮件、Web 数据传输；而 UDP 适合对时效性要求高、数据丢失不影响整体体验的场景。

6. HTTP vs HTTPS

- **HTTP** 是不加密的传输协议，数据在客户端和服务端之间以明文方式传输，存在被窃取或篡改的风险。
- **HTTPS** 是 HTTP 的加密版本，使用 SSL/TLS 协议对传输内容进行加密，保证了数据的安全性。HTTPS 在客户端和服务端之间建立了一个安全的加密通道，数据传输过程更加安全。

7. Webpack 热更新原理

Webpack 的 **热模块替换（HMR）** 允许在不刷新整个页面的情况下更新模块。其工作原理如下：

1. 在开发环境中，Webpack 会监听文件的变化。
2. 当文件发生变化时，Webpack 通过 WebSocket 与浏览器通信，发送更新后的模块。
3. 浏览器通过 JavaScript 注入这些更新模块，替换掉旧的模块，同时保留应用的状态，不重新加载整个页面。

8. Tree Shaking

Tree Shaking 是 Webpack 中的一项优化技术，通过消除未使用的代码来减小打包后的文件体积。它依赖于 ES6 的模块机制，因为 ES6 模块是静态分析的，这意味着编译时可以确定哪些模块被实际使用。

使用条件：

- 必须使用 ES6 的 `import` 和 `export` 语法，而不是 CommonJS (`require`) 语法。
- 必须启用 Webpack 的 `production` 模式，因为 Tree Shaking 只在生产环境中生效。

9. JSX 与 JS 的区别

- **JSX** 是一种语法扩展，允许我们在 JavaScript 中书写类似 HTML 的代码，主要用于 React 开发。JSX 最终会被编译为 JavaScript 函数调用（通常是 `React.createElement`）。
- 例子：

```
1 const element = (  
2   <div>
```

```
3     <h1>Hello, world!</h1>
4     <p>This is a JSX example.</p>
5 </div>
6 );
```

- **JSX 只有一个父节点** 是因为 React 元素的结构是基于树的，每个组件必须返回一个根节点。

10. 最长回文子串（算法题）

最长回文子串问题可以使用中心扩展法或动态规划法来求解。下面是一个使用中心扩展法的示例代码：

```
1 function expandAroundCenter(s, left, right) {
2     while (left >= 0 && right < s.length && s[left] === s[right]) {
3         left--;
4         right++;
5     }
6     return right - left - 1; // 回文长度
7 }
8
9 function longestPalindrome(s) {
10     if (s.length < 1) return "";
11     let start = 0, end = 0;
12     for (let i = 0; i < s.length; i++) {
13         const len1 = expandAroundCenter(s, i, i); // 奇数长度回文
14         const len2 = expandAroundCenter(s, i, i + 1); // 偶数长度回文
15         const len = Math.max(len1, len2);
16         if (len > end - start) {
17             start = i - Math.floor((len - 1) / 2);
18             end = i + Math.floor(len / 2);
19         }
20     }
21     return s.substring(start, end + 1);
22 }
```

此算法通过每个字符作为回文的中心，向两侧扩展，寻找最长的回文子串。